Table 7.1 Initialization and teardown hooks used by different BDD tools *(continued)*

Stage	JBehave	Cucumber/ JVM	Lettuce	SpecFlow
After each feature or story	`@AfterStory`	-	`after_feature`	`[AfterFeature]`
After the test suite	`@AfterStories`	-	`after_all`	`[AfterTestRun]`
Before a given tag	-	`@Before`	`before_tag`	`[BeforeScenario]`
After a given tag	-	`@After`	`after_tag`	`[AfterScenario]`

Let's look at how these hooks work in each of these tools.

INITIALIZATION HOOKS IN JBEHAVE

In JBehave you can use the `@BeforeScenario` or `@BeforeStory` annotation to reset the database before each scenario or story, whereas `@BeforeStories` lets you reset the database once at the start of the test suite.

For example, suppose you've written a `Database` class that's responsible for setting up your test database. To reset the database before each scenario, you could write something like this:

```
@BeforeScenario
public void initializeDatabase() {
    Database.initialize();
}
```

JBehave will call this method before each scenario.

A custom class you write to initialize the test database.

The `@BeforeStory` annotation can also be used to intervene before the execution of a row in a table-driven test, as shown here:

```
@BeforeScenario(uponType = ScenarioType.EXAMPLE)
public void prepareDatabaseForANewExample() {...}
```

This method will be executed before each row in a table-driven scenario.

There are also symmetrical `@AfterScenario`, `@AfterStory`, and `@AfterStories` annotations that can be used to tidy up the database afterwards.

INITIALIZATION HOOKS IN CUCUMBER-JVM

Other BDD frameworks support similar features. For example, Cucumber-JVM has a more limited set of annotations, providing the `@Before` and `@After` annotations to mark methods that will be called before and after each scenario:

```
@Before
public void initializeDatabase() {
    TestDatabase.initialize();
}
```

Cucumber calls this method before each scenario.